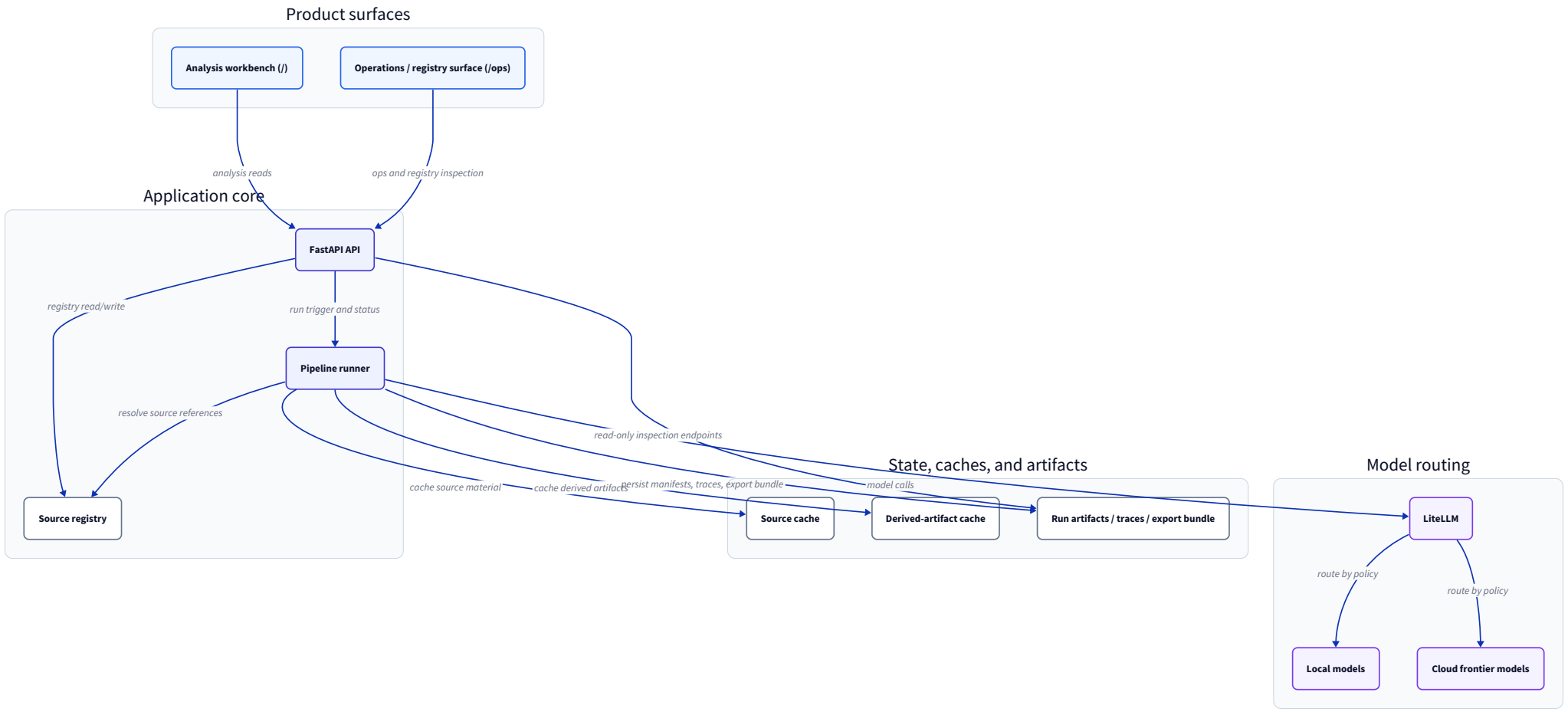
Judit — Technical architecture

Architecture deck for engineers and integrators.

Source-of-truth: `docs/architecture/system-overview.md`, `docs/architecture/pipeline-overview.md`, `docs/reference/artifacts.md`, ADRs under `docs/decisions/`.

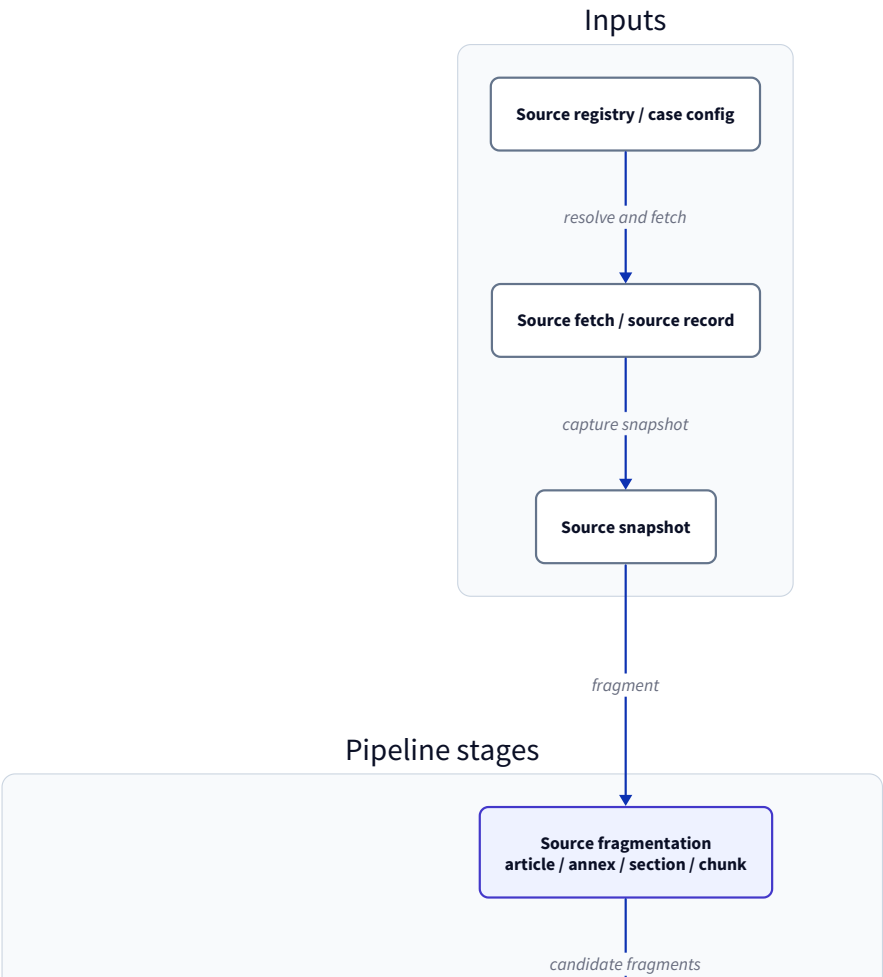
Architecture at a glance



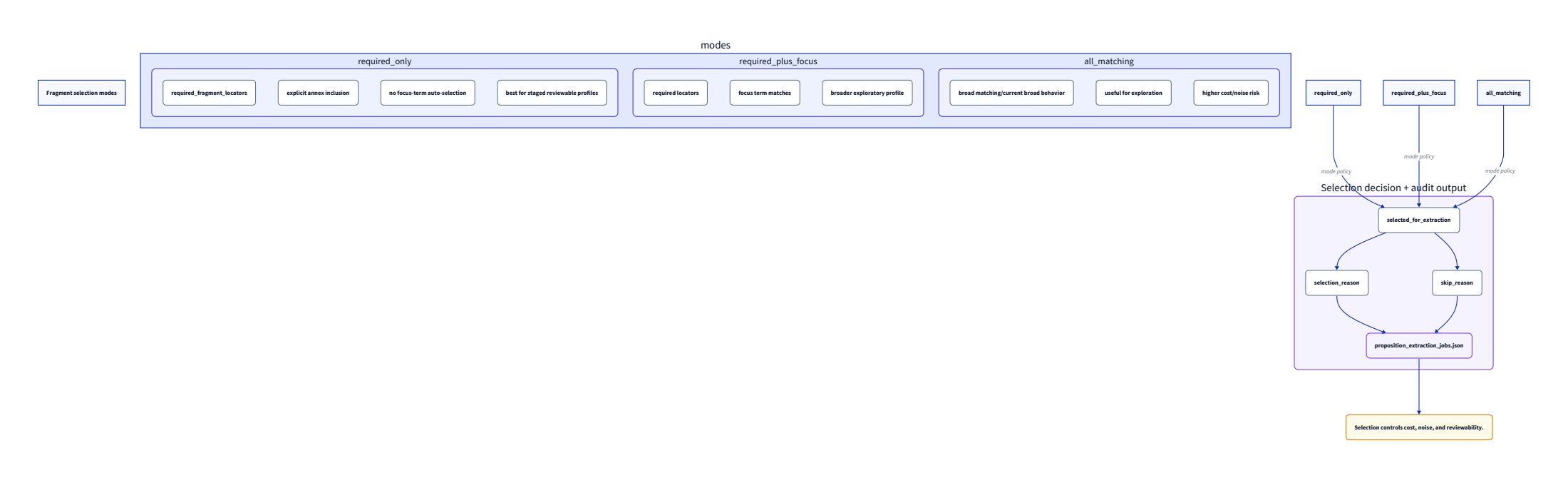
Layers

1. **Pipeline and storage** — ingest sources, run staged processing, persist inventories, artefacts, traces, and quality summaries under `dist/static-report/`.
2. **LLM gateway** — LiteLLM/OpenAI-compatible endpoint routes controlled model calls for the small set of assisted stages under policy (see ADR-0019).
3. **Analysis and operations surfaces** — read-mostly workbench (`/`) and operations/registry inspector (`/ops`); selected controlled mutation paths for registry, review decisions, async run jobs, repair, and dev reset.
4. **Docs and export assets** — published narratives and bundles built

Pipeline (v2)

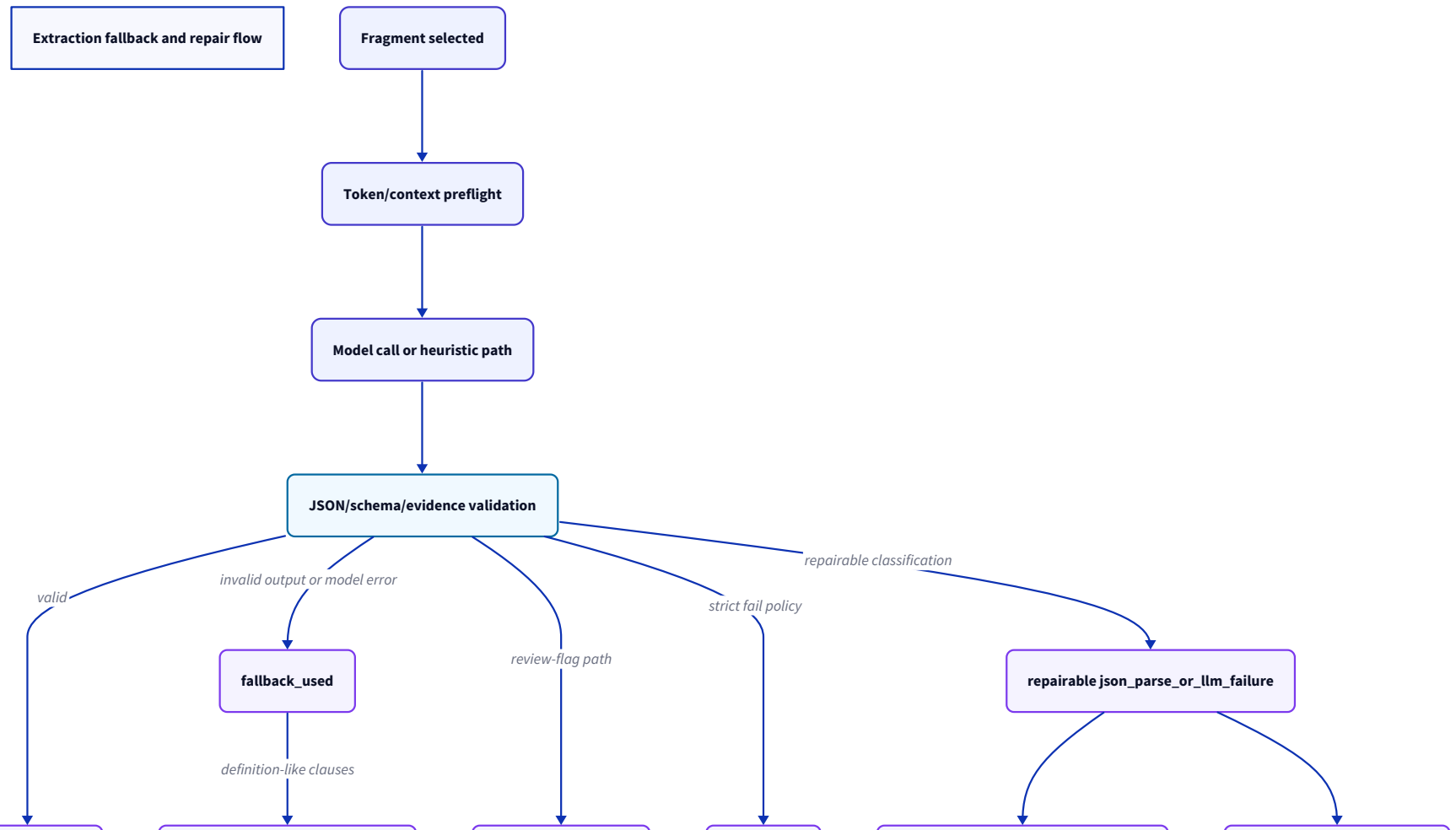

Inputs
Source registry / case config
resolve and fetch
Source fetch / source record
capture snapshot
Source snapshot
fragment
Pipeline stages
Source fragmentation
article / annex / section / chunk
candidate fragments

Fragment selection modes

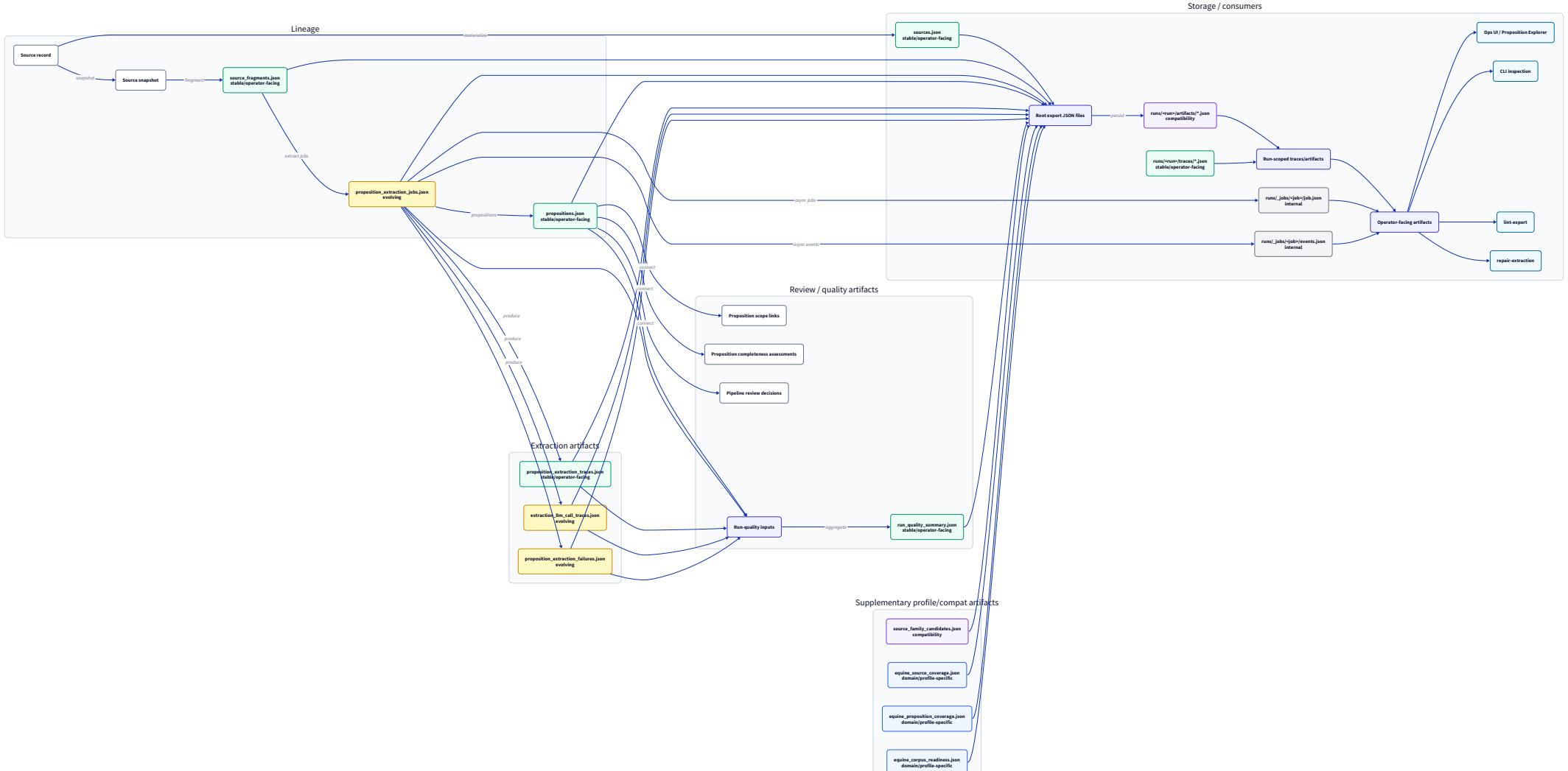


`required_only`, `required_plus_focus`, `all_matching` — selection controls cost, noise, and reviewability. Selection/skip audit fields are written to `proposition_extraction_jobs.json` for every run.

Extraction repair flow



Run artifacts



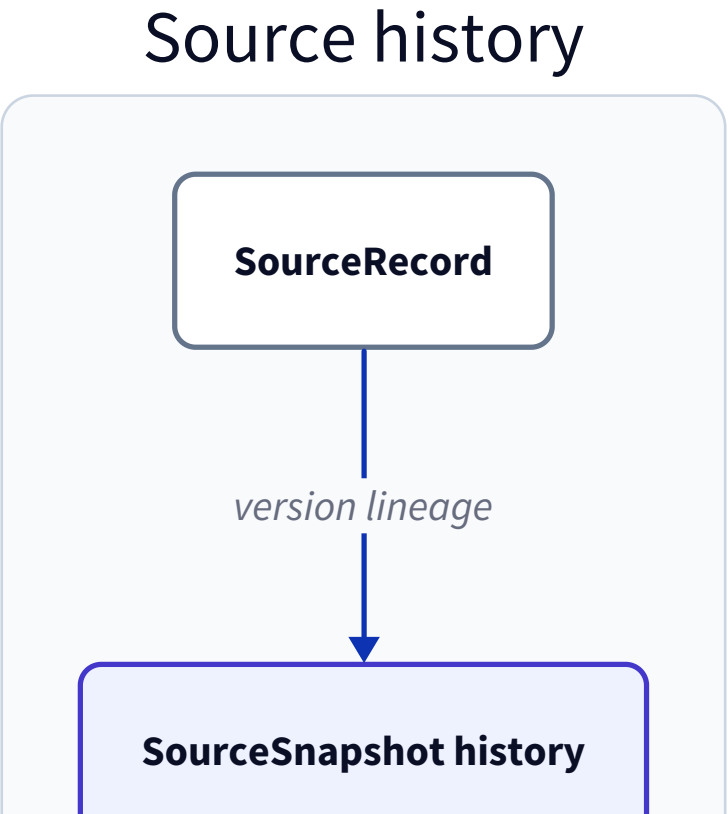
Proposition identity (ADR-0018)

- `Proposition.id` is opaque, stable machine identity.
- `proposition_key` carries source lineage: `<instrument-id>:<fragment-locator>:pNNN` when derived from registered sources.
- `label` / `short_name` are human-readable; `slug` is for routing only and not durable identity.

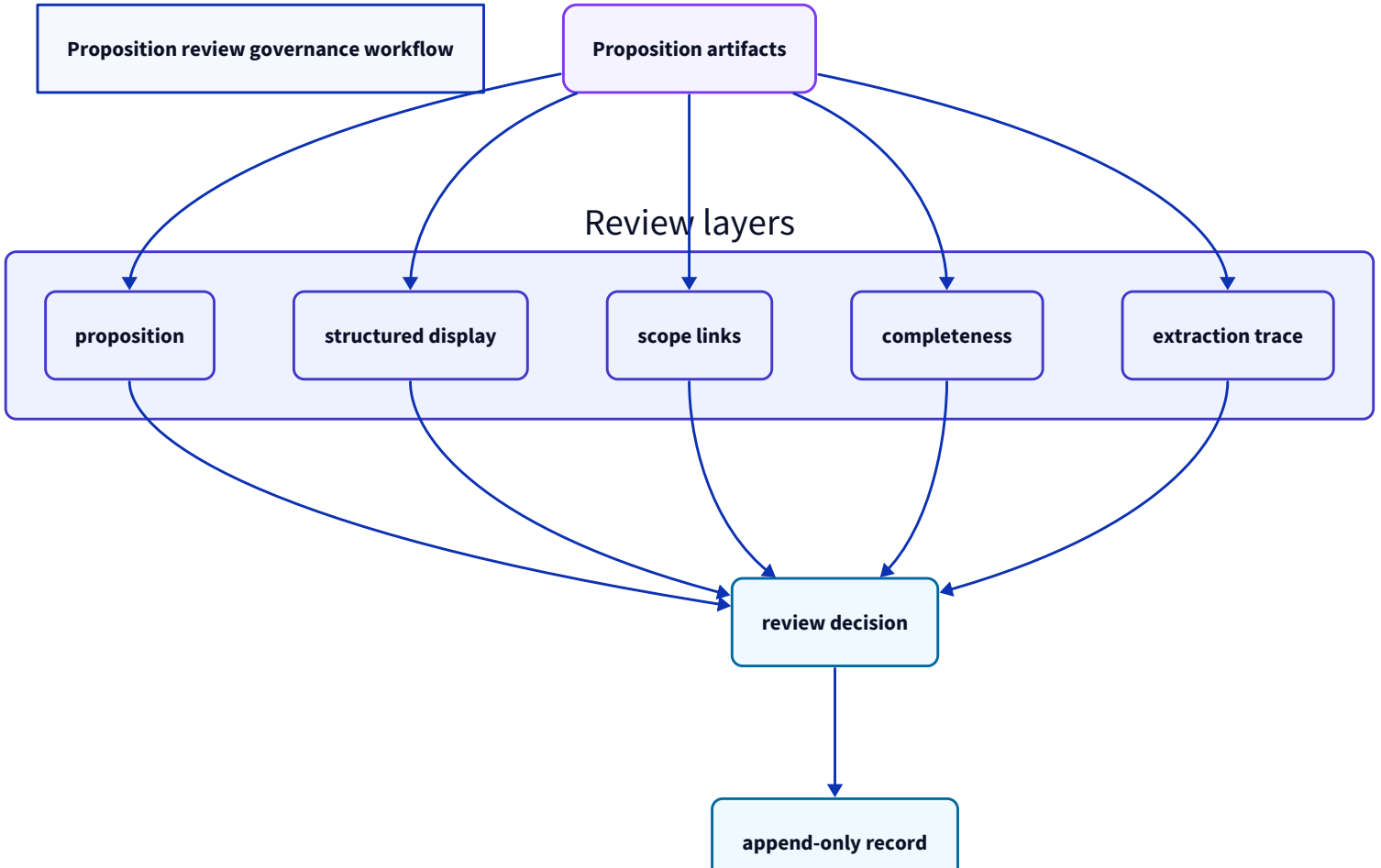
Model usage strategy (ADR-0019)

- Deterministic-first pipeline; models only in explicitly defined stages.
- `heuristic` (deterministic / no model), `local`, `frontier` extraction modes.
- Traces persist in run artifacts: `proposition_extraction_traces.json`, `proposition_extraction_jobs.json`, `extraction_llm_call_traces.json`.
- `extraction_fallback`: `fallback` / `mark_needs_review` / `fail_closed`.
- `model_error_policy`: `continue_with_fallback` / `continue_repairable` / `stop_repairable`.

Temporal audit



Review governance



Operational/runtime truth

- Run artifacts persist under `dist/static-report/runs/<run-id>/`.
- Source registry state persisted separately, inspectable/refreshable via API.
- Async jobs: `runs/_jobs/<job>/job.json` + `events.json` for status and progress timeline.
- Read paths via `/ops/*`; write paths only on documented mutation endpoints.

Boundaries kept stable

- Export bundle remains the renderer-facing boundary so future static renderers consume validated output without owning business logic.
- Stable vs evolving vs domain-specific artifact contracts are literal — pin behavior accordingly.

ADRs to read

- ADR-0005 — Source-first ingestion
- ADR-0006 — Proposition-level comparison unit
- ADR-0007 — Export bundle / renderer boundary
- ADR-0010 — Human review in core workflow
- ADR-0013 — Traceable pipeline run artifacts
- ADR-0014 — Proposition-first architecture
- ADR-0017 — Operational history API surface
- ADR-0018 — Proposition identity and naming
- ADR-0019 — Model usage strategy